Factor Search in HQIV_LEAN: Algorithms, Combinatorial Certificates, and Formalized Proofs

HQIV_LEAN repository (machine-checked fragments)

May 1, 2026

Abstract

This note records the *algorithmic* structure of two HQIV factor-search drivers ("integrated" sparse OSH and "monolithic" geometric factorizer) and summarizes what is actually proved in the Lean 4 library `HQIV_LEAN`: sound divisibility certificates, gcd/remainder equivalence, sparse-register list algebra, explicit per-step charging schemes, and a polynomial *template* bound for a combined integrated cost model. We explicitly separate these results from open claims: no theorem here establishes that either driver finds a nontrivial factor on every composite input, nor that wall-clock time scales as a particular function of $\log n$ without additional machine hypotheses.

Contents

1 Introduction

The repository implements classical search loops paired with Lean mirrors. The mathematical content that is *proved* falls into three classes:

(i) **Sound certificates:** once an integer passes a modular test, the factorization identities recorded in Lean follow from standard arithmetic.

(ii) **Combinatorial bookkeeping:** sparse simulation steps expand and prune lists; Lean bounds list lengths and sums of lengths.

(iii) **Cost scaffolding:** under explicit definitions of "fuel" and per-step charges, Lean proves inequalities such as a degree-6 polynomial bound in $\lfloor \log_2(n+1) \rfloor + 1$.

What is *not* proved (unless explicitly stated as a conjecture or roadmap item) includes completeness of search, correctness of floating-point angle rounding, and primality of factors returned by probabilistic engines.

2 Algorithms (informal)

2.1 Integrated driver (`hqiv_osh_integrated_driver.py`)

At a high level the script:

- Peels factors of two from the input, working on an odd core.
- Builds a sparse harmonic register (cutoff L, basis size $(L+1)^2$).
- Runs sparse gate steps: expand support, apply a digital gate, detect flipped indices, prune to flipped support (the "sparse Shor-style" spine).
- Maintains phase / angle channels that map indices to integer candidates; on a hit, checks classical divisibility.

The Lean module `HQIVOSHIntegratedFactorDriver` names the discrete maps (`rootCap`, `idxAngleCandidate`, `wrapIdx`, ket fallback) and packages divisor witnesses after peeling.

2.2 Monolithic geometric factorizer (`monolithic_geometric_factorizer.py`)

The monolithic script:

- Uses dynamic decimal precision tied to the bit length of the odd core.
- Maps rational turns a/n to integer candidates via a root cap $\max(2, \lfloor \sqrt{n} \rfloor)$.
- Applies multiplication gates with inverse strength keyed off register bit sizes ("lowest register" variant in Lean).
- Prunes candidate lines by an age threshold derived from $\text{bitlen}(|c-n|) + 1$.
- Caps the number of parallel registers at cube-root scale $\min(128, \lfloor n^{1/3} \rfloor)$ (after peeling twos).

- Accepts a factor only on a purely integer test: $1 < d <$ odd and odd mod $d = 0$.

The Lean file `MonolithicGeometricFactorizer` mirrors these discrete parameters and isolates real-number drift resources (`MonolithicFracResources`) as a *template* for aligning floating-point $\theta/(2\pi)$ with floor slots.

3 Divisibility certificates and GCD

3.1 Odd-core witness

Definition 3.1 (Odd-core factor witness)**.** Given odd $n \in \mathbb{N}$, an *odd-core factor witness* consists of $d \in \mathbb{N}$ with $1 < d < n$ and $d \mid n$.

Lean encodes this as `OddCoreFactorWitness` (`HQIVOSHIntegratedFactorDriver`). Reconstruction $n = d \cdot (n/d)$ is `OddCoreFactorWitness.reconstructs`.

3.2 Peeling powers of two

If orig $= 2^k \cdot$ odd and $d \mid$ odd, then $2^k d \mid$ orig (`two_pow_mul_dvd_of_dvd_odd`).

3.3 GCD and division (`FactorDivisibilityBridge`)

Proposition 3.2 (Division and quotient)**.** *For all $d, n \in \mathbb{N}$, $d \mid n$ if and only if $n = d \cdot (n/d)$.*

Proposition 3.3 (GCD as the same certificate class)**.** *If* $1 <$ gcd$(x,$ odd$) <$ odd*, then* gcd$(x,$ odd$)$ *yields an odd-core witness. If already $d \mid$* odd*, then* gcd$(d,$ odd$) = d$*.*

Remark 3.4. These statements are pure arithmetic; they do *not* guarantee that a particular random x produces a nontrivial gcd. A concrete counterexample is $\gcd(2, 15) = 1$ while $3 \mid 15$ (`gcd_probe_can_miss_nontrivial_factor`).

3.4 Monolithic acceptance

Definition 3.5. `monolithicAcceptsDivisor`$(n, d) \equiv 1 < d < n$ and $n \bmod d = 0$.

Proposition 3.6. *`monolithicAcceptsDivisor`*$(n, d) \Leftrightarrow 1 < d < n$ *and* $d \mid n$ *(`monolithicAcceptsDivisor_iff`*

Proposition 3.7 (Compositeness)**.** *If $1 < n$ and `monolithicAcceptsDivisor`(n, d), then n is not prime (`not_prime_of_monolithicAccepts`).*

4 Sparse OSH oracle: list algebra

Let a `SparseRegister` be a finite list of indexed amplitudes. One step:

1. `causalExpandSupport` doubles support in length (horizon-causal expansion).
2. `applyGateSparse` maps to length $2|r|$ (`applyGateSparse_length_eq_two_mul`).
3. `detectFlippedKets` lists indices that changed between "before" and "after".
4. `pruneToFlipped` filters to a flipped index set; length never increases (`pruneToFlipped_length_le`).

Proposition 4.1 (Integrated step length)**.** *After evolve then prune,* $|\text{prune}| \le 2 \cdot |r|$ *(`integrated_pruned_sparse`*

Proposition 4.2 (Triple sum bound)**.** *With $e =$ `applyGateSparse`(g, r) and $f =$ `detectFlippedKets`(r, e),*

$$|r| + |e| + |f| \le 6|r|$$

(`sparse_step_list_sum_le_six_mul` in `FactorSearchCostModel`).

5 Integrated driver maps and bounds

5.1 Root cap and angle candidate

$\texttt{rootCap}(n) = \max(2, \lfloor\sqrt{n}\rfloor)$, always ≥ 2.

The discrete angle candidate (for basis b and index `idx`) satisfies

$$\texttt{idxAngleCandidate}(n, b, \texttt{idx}) \leq 2 + \texttt{idx} \cdot \texttt{rootCap}(n)$$

(`idxAngleCandidate_le_two_add_idx_mul_rootCap`).

5.2 Wrapped indices

$\texttt{wrapIdx}(L, \texttt{idx}) < (L+1)^2 = \texttt{sparseBasisCard}(L)$.

5.3 Ket fallback alignment

The integrated ket fallback agrees with the gate-frontier cofactor map on wrapped indices (`ketLinearFallback_eq_cofactorCandidate`), and when $2 \leq \texttt{qSpan}(n)$, candidates stay inside the Q-shell bound (`ketLinearFallbackCandidate_le_qSpan`).

6 Cost model and polynomial template

6.1 Bit-scale parameter

$\texttt{inputLog2}(n) := \lfloor \log_2(n+1) \rfloor$ and $d := \texttt{inputLogDim}(n) = \texttt{inputLog2}(n) + 1$.

6.2 Default harmonic cutoff

$\texttt{defaultHarmonicCutoff}(n) = \max(4, \lfloor\sqrt{n}\rfloor)$, hence $L + 1 \leq 5 + \lfloor\sqrt{n}\rfloor$ and basis cardinality $(L+1)^2 \leq (5 + \lfloor\sqrt{n}\rfloor)^2$ for this default.

6.3 Combined per-step cap

The file `FactorSearchCostModel` defines a conservative combined cap `integratedCombinedPerStepCap` that sums:

- a sparse list charge modeled as $6 \cdot d^3$ in terms of $d = \texttt{inputLogDim}(n)$;
- a phase-channel charge with history length bounded by fuel and extract cost f^2.

Theorem 6.1 (Per-step domination)**.** *integratedCombinedPerStepCap*$(n) \leq 12 \cdot d^4$ *(integratedCombinedPerS*

6.4 Total work under polylog fuel

Let $\texttt{fuelPolyLogSq}(n) = d^2$. If $\texttt{fuel} \leq \texttt{fuelPolyLogSq}(n)$, then naive total work charged at `integratedCombinedPerStepCap` satisfies

$$\texttt{fuelTotalWork}(\texttt{fuel}, \texttt{integratedCombinedPerStepCap}(n)) \;\leq\; 12 \cdot d^6$$

(`total_integrated_work_le_twelve_mul_inputLogDim_pow_six`).

Remark 6.2. This is a *rigorous inequality in the cost model* once fuel and charges are instantiated as above. It is not, by itself, a wall-clock theorem: real implementations allocate dense intermediates, use floating-point, and may terminate earlier or later than the fuel model.

7 Monolithic discrete policy (selected lemmas)

7.1 Cube-root floor

`floorCbrt`(n) is the greatest r with $r^3 \le n$; it satisfies the standard maximality properties (`floorCbrt_cube_le`, `floorCbrt_lt_succ_cube`).

7.2 Dynamic precision

`dynamicPrecisionDigits`$(n) \le 2 \cdot$`inputLog2`$(n)+11$ (`dynamicPrecisionDigits_le_two_mul_inputLog_add_el`

7.3 Prune age threshold

For candidate c and target n, write $d = \mathrm{dist}(c, n)$ and define bit-length of d as 0 if $d = 0$ else $\lfloor \log_2 d \rfloor + 1$. Then `pruneAgeThreshold`$(n, c)$ is this bit-length plus 1.

For $c \le n$, `pruneAgeThreshold`$(n, c) \le$ `inputLog2`$(n)+3$ (`pruneAgeThreshold_le_inputLog_add_three`).

7.4 Mod-periodicity certificate

If $f(t+p) \equiv f(t) \pmod m$ for all t, then residues at ages reduce to $f(\text{age} \bmod p)$ (`mod_residue_eq_mod_age`) and equal ages with the same remainder mod p yield equal residues (`age_redundant_mod_obs_of_global_period`)

8 Counterexamples (boundary effects)

Concrete arithmetic facts proved with `native_decide`:

- Distances $\mathrm{dist}(993, 1000) = 7$ and $\mathrm{dist}(992, 1000) = 8$ but different `pruneAgeThreshold` values 4 vs. 5 (`pruneAgeThreshold_jumps_across_dist_boundary`).
- Register bit-size proxies differ across 7 vs. 8 (`registerBitSize_jumps_at_eight`).
- Inverse lowest-register strength differs for pairs $(7, 100)$ vs. $(8, 100)$ (`strengthInvLowestRegister_jumps_w`

9 What remains outside the formal proof

G1. Search completeness: no Lean theorem shows that either driver encounters a divisor for every composite n under stated caps.

G2. Period → divisor: the Shor-style bridge from Fourier/period peaks to gcd factors is not closed inside these files.

G3. Floating-point: adequacy of real drift budgets (`FracDriftAdequateResources`) is a template; exact Python `mpmath` semantics are not embedded.

G4. Primality output: probable-prime tests in scripts are not theorems.

Acknowledgments

Statements in this note are intended to track the Lean modules `Hqiv.Geometry.FactorDivisibilityBridge`, `Hqiv.Geometry.HQIVOSHIntegratedFactorDriver`, `Hqiv.Geometry.MonolithicGeometricFactorizer`, `Hqiv.Geometry.FactorSearchCostModel`, and `Hqiv.QuantumComputing.OSHoracle`. When in doubt, the Lean source is authoritative.

References

[1] The Mathlib community, *Mathlib4*, `https://github.com/leanprover-community/mathlib4`.